This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

GR-Gauge: Cost-efficient Training Configuration By Gauging the Gradient Redundancy

Guanjie Wang
Zhiyuan College, Shanghai Jiao Tong University

Chen Chen
Shanghai Jiao Tong University

Abstract

The recent success of artificial intelligence motivates many non-professional users to train their own models. Those users often resort to cloud training services, seeking to obtain a sufficiently accurate model at a modest cost, for which properly setting up the learning rate and batch size is crucial. While various Hyper-parameter Optimization (HPO) methods have been proposed in that regard, they largely act based on heavy-weight validation signals, being inefficient in the overall cost. We find that the model training process can be viewed as a two-dimensional voting process—with gradients for different iterations and from different samples; moreover, to attain cost-efficient training is to ensure that the gradient redundancy is within a proper range which is similar across diverse models. Based on that insight, we further introduce GR-Gauge, a general method that gauges the gradient redundancy to instruct HPO decisions like configuration searching and trial termination. Extensive experiments demonstrate that GR-Gauge can help attain near-optimal accuracy in much less time than existing methods.

1. Introduction

Artificial intelligence has revolutionized various application domains like image recognition [33] and text generation [22, 54], and many non-professional users now step in by developing their own neural network models [4, 6]. Those non-professional users, like amateurs and start-ups without dedicated GPU clusters, are usually cost-sensitive; they often resort to cloud services for model training [1, 3, 5], expecting to obtain a model with satisfiable accuracy at a modest cost [2]. For good cost-to-accuracy performance, it is crucial for the service provider to properly set the learning rate and batch size—two most classical hyper-parameters—for general user models.

To find proper hyper-parameters of a model training process, a typical methodology is Hyper-Parameter Optimization (HPO). HPO means to automatically find the optimal hyper-parameters in model training with a series of trial runs [8, 9, 11]. It essentially involves making two decisions (searching and scheduling): what hyper-parameters to search in the next trial—with methods like grid search [8] or Bayesian Optimization [11], and how many resources to allocate for each trial—with early-termination methods like Successive Halving [24] and Hyperband [30]. Nonetheless, existing HPO methods heavily rely on the validation metrics like accuracy when making such searching and scheduling decisions, which, although general, suffers low efficiency due to high validation overhead and low decision quality.

In fact, when optimizing learning rate and batch size, we can leverage the gradient information to attain higher efficiency. Specifically, we notice that the model training process can be viewed as a *two-dimensional voting process*: in the *temporal* dimension, the (global) gradients in different iterations continuously vote for the parameter updating direction; in the *spatial* dimension, the (local) gradients from different samples vote for the overall parameter updating direction in each iteration. To make such voting process cost-efficient, we need to avoid trivially consistent or counteracting voting. That is, we need to ensure that the training process is in a *healthy* state—with a modest gradient redundancy level. In that regard, we propose two metrics, *temporal gradient redundancy* and *spatial gradient redundancy* (GR_T and GR_S), to help quantify the healthiness level of a model training process. A series of testbed measurements further show that, the early-stage values of GR_T and GR_S can indeed reflect the cost-efficiency (i.e., healthiness) of the entire model training process; meanwhile, their ranges indicating healthy training are *stable across diverse models*. Such a healthy range can be leveraged as a general gauge to guide the HPO process.

In this paper, we propose GR-Gauge, a generic method that attains cost-efficient HPO by gauging the gradient redundancy. Our theoretical analysis shows that there exists a clear correlation between (GR_T, GR_S) and (learning rate, batch size); with the healthy (target) range of (GR_T, GR_S), we accelerate the HPO process with three measures. First, we can attain efficient early-stopping by periodically checking whether the GR_T and GR_S values are within the healthy range. Second, based on the gap between the measured (GR_T, GR_S) values and the targeted values indicating

healthy training, we can determine the next configuration to explore. Third, we allow the intermediate state of an aborted trial be reused by the next trial—if the (GR_T, GR_S) value can timely return to the healthy range after continuously training under the new setup; this can reduce resource wastage consumed by trial failures.

We evaluate the performance of GR-Gauge by comparing it with popular HPO methods over diverse models. End-to-end measurements show that it can reduce the total time to reach a near-optimal model performance by over 80%. Ablation studies further confirm that it is indeed necessary to apply the healthiness guidance to each of three aspects (i.e., configuration-search, early-termination and state-reuse). Moreover, the additional overheads brought by enforcing GR-Gauge is much less than the performance benefits.

2. Background

2.1. Automatic Training Configuration in TaaS

Due to the high cost to maintain dedicated GPU servers, cost-sensitive users with model training demands often resort to public cloud platforms that provision *Training-as-a-Service* (TaaS) [1, 3, 5]. To use such services, a tenant submits its own model, dataset and validation metric to the service provider—with all the remaining issues (like hyper-parameter configuration, resource provisioning) handled by the service provider [2]. In such scenarios, the users expect their models to promptly reach a high accuracy at a low cost, and the provider expects to have a high resource utilization when serving many users concurrently.

Facing diverse model training tasks, the service provider needs to automatically configure the training process in a user-transparent manner—in this paper we focus on *learning rate* and *batch size*, which are the most significant hyper-parameters in model training. In the literature, it has been shown that setting them improperly (either over-large or over-small) would lead to poor training performance (in terms of *time-to-target-accuracy*) [16, 32, 51]. Yet, it remains a challenging task for the service provider to properly set up the learning rate and batch size for arbitrary models.

Specifically, we focus on promptly finding a proper (learning rate, batch size) setup within the early training period. While some online-tuning methods [12, 25, 39, 47] have proposed to gradually scale the learning rate or batch size in an online manner during the entire training process, their well-behaving is still dependent to the initial hyper-parameter setup. In fact, existing works [57, 58] have identified the existence of the *critical learning period* in model training: if the initial hyperparameter setup is improper and not remediated timely, the ultimate model training performance would be permanently damaged (regardless of how the hyper-parameters are tuned later). Therefore, to realize cost-efficient TaaS, it is in critical need to—for general models—properly initialize the learning rate and batch size.

2.2. Related Works

In the literature, Hyper-Parameter Optimization (HPO) [9, 11] is often adopted to automate hyper-parameter configurations. HPO algorithms typically involve two key parts: *searching*—which selects the next hyperparameter candidate to evaluate, and *scheduling*—which determines how many resources are invested for evaluating each candidate. Regarding the *searching* part, typical algorithms include *uninformed searching* (such as grid search and random search) [8] and *Bayesian Optimization* (BO) [11, 50]. BO uses a surrogate model (e.g., Gaussian process [17, 52]) to guide the selection of candidate configurations. Regarding the *scheduling* part, early stopping mechanisms used in multi-fidelity schemes—including Successive Halving (SH) [24] and Hyperband (HB) [30], choose to early-terminate poorly performing configurations (based on the validation metrics like accuracy) to save resources for more promising candidates. Further, BOHB [15] combines BO and HB to harvest the benefits in both sides—fast candidate identification (from BO) and light-weight per-candidate exploration (from HB).

In particular, existing HPO methods prevalently rely on *external validation metrics* (e.g., accuracy) to make scheduling and searching decisions, so as to be generally applicable to diverse hyper-parameters (e.g., system hyper-parameters like learning rate, and model architecture hyper-parameters like layer number). However, although being hyper-parameter generic, using such validation metrics for HPO is deficient in three aspects.

(1) First, with large validation datasets, calculating the validation metric itself would be quite time-consuming. For example, when training a BERT model on an RTX 2080 Ti GPUs, it takes around 250 seconds to conduct a single run over the validation dataset to calculate the F1 score. *(2) Second*, the validation metrics may be inaccurate for making scheduling decisions of HPO (e.g., identify improper setups for early-termination). While existing HPO methods assume good shape of the validation metric curve in training [24], some recent works [37, 46] have found that such an assumption may not hold in practice—good hyper-parameters may also perform badly at the early training stages. *(3) Third*, external validation metrics are not able to guide efficient hyper-parameter searching. While a low trial accuracy can indicate an improper configuration, it fails to suggest which subsequent searching direction is more promising.

In fact, when specifically optimizing learning rate and batch size, we can trade hyper-parameter generality for HPO efficiency, by leveraging the internal training signals like gradient statistics. Gradient-based signals in the model training process are always readily-observable for the TaaS service provider. In this paper, we show that such gradient-based information—by suggesting a precise exploration direction

and timely stopping a diseased trial—can help attain efficient HPO specifically for learning rate and batch size.

3. Motivation

In this section, we explore how gradient-based feedback signal can be exploited to diagnose how efficient the model training process is under the current hyper-parameter setup.

3.1. The Metric of Gradient Redundancy

Model training: a two-dimensional voting process. Given a machine learning model, the objective of training is to find the optimal parameters that minimize the loss function, which can be expressed as $\omega_{t+1} = \omega_t - \gamma_t g_t$, where ω_t are the parameters at iteration t, g_t is the gradient of loss function (calculated from a set of samples), and γ_t is the learning rate. We note that the model training process can be viewed as conducting two-dimension gradient voting process.

First, from a *temporal* perspective, the training process can be viewed as the accumulation of gradient updates over iterations. The final parameters ω can be expressed as $\omega = \omega_0 - \sum_t \gamma_t g_t$. Each iteration t contributes a gradient g_t, and the training process aggregates these contributions over time. In this perspective, each iteration "votes" its gradient to guide the optimization. Second, from a *spatial* perspective, the training process involves aggregating the local gradients from multiple samples (organized in mini-batches) within a single iteration. The parameter update at iteration t can be expressed as $\omega_{t+1} = \omega_t - \frac{1}{n}\gamma_t \sum_{i=1}^{n} g_{t_i}$, where each sample i computes a local gradient g_{t_i}, and the applied update is the average of these gradients. In this perspective, each sample "votes" its local gradient.

Gradient voting redundancy: an effective angle to percept training efficiency. We now think out of the box. With the gradient-voting point of view, we note that the efficiency of training depends on the redundancy of 'votes' (i.e., gradient) among different voters. On the one hand, when the gradients from different voters demonstrate significant redundancy, it implies that the training process is reiterating the same information, resulting in inefficient use of computational resources. On the other hand, if the gradients are highly inconsistent, it leads to parameter oscillations caused by conflicting updates, also resulting in a significant reduction in training efficiency. Both training situations can thus be deemed "unhealthy", suffering compromised training efficiency (in terms of the accuracy enhancement from per processed sample). Therefore, to ensure efficient training is to maintain a *proper degree* of gradient redundancy in both dimensions (to be confirmed later in Fig. 1).

Yet, it remains unclear what specific level of gradient redundancy is "proper". To find out, we first propose two metrics that respectively quantifie the gradient redundancy level in the temporal and spatial dimension.

Table 1. Models and datasets used in our experiments.

Task	Model	Dataset	Validation
Recommendation	NeuMF [21]	MovieLens [19]	Hit Rate
Image Classification	ResNet18 [20]	Cifar10 [28]	Accuracy
Question Answering	Bert [13]	SQuAD [48]	F1 Score
Text to image	Stable-diffusion-v1-5 [49]	Naruto BLIP captions [10]	Loss
Instruction Fine-tuning	Llama-3.2-3B-Instruct [42]	FineTome-100k [44]	Loss

Quantitative metric for depicting temporal and spatial gradient redundancy. In quantifying the gradient redundancy, we maintain two separate metrics for the two orthogonal voting dimensions: GR_T—the temporal-dimension metric which depicts the consistency level among the global gradients from different iterations, and GR_S—the spatial-dimension metric which depicts the consistency level among the local gradients from different samples. To be specific, motivated by gradient-based adaptive optimizers like AdaGrad [14] and Adam [27], we have the following definitions.

Definition 1 (Temporal Gradient Redundancy)**.** *The temporal gradient redundancy metric GR_T of the training at iteration t is defined as*

$$GR_T(t) = \frac{d \cdot \hat{v}_t}{\hat{m}_t{}^2}, \tag{1}$$

where d is the dimension of the g_t, and $\hat{m}_t$ and $\hat{v}_t$ respectively represent[1] *the bias-corrected first and second moment estimation (β is constant smooth factor, which is defaulted to 0.9 in this paper). That is, $\hat{m}_t = m_t/(1-\beta^t)$ and $\hat{v}_t = v_t/(1-\beta^t)$, where $m_t = (1-\beta)\cdot g_{t-1} + \beta \cdot m_{t-1}$ and $v_t = (1-\beta)\cdot g_{t-1}^2 + \beta \cdot v_{t-1}$.*

Definition 2 (Spatial Gradient Redundancy)**.** *Let g_{t_i} be the sampled gradient of sample i at iteration t, then the spatial gradient redundancy metric GR_S of the training at iteration t is defined as:*

$$GR_S(t) = \frac{||\sum_{i=1}^{B} g_{t_i}^2||_1}{||\sum_{i=1}^{B} g_{t_i}||_2^2}, \tag{2}$$

where B is the total number of samples processed in an iteration (i.e., global batch size).

3.2. A Model-generic Healthiness Criterion

Given the above two metrics, we then show that 1) *the (GR_T, GR_S) value during the critical period is a crucial indicator of the overall training efficiency*, and 2) *the ideal metric values that indicate healthy training are stable across diverse models*.

As shown in Table 1, we select five representative models, and train them under different (learning rate, batch size)

[1] $\hat{m}_t$ and $\hat{v}_t$ are coordinate-wise; we first calculate the redundancy metric for each coordinate and then calculate the global average.

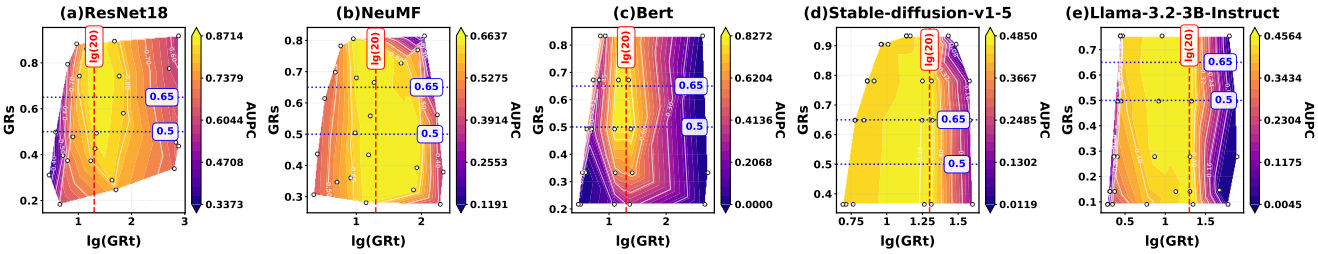


Figure 1. The relationship between (GR_T, GR_S) and the ultimate AUPC performance for all models described in Table 1.

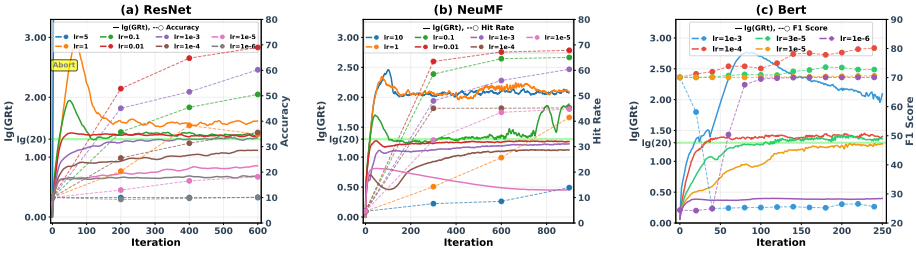


Figure 2. GR_T and validation metrics across iterations under different learning rate configurations in critical period.

configurations; each model is trained for a fixed number of epochs (20, 60, 2 and 3, respectively, which are sufficient for convergence). In particular, to uniformly describe the overall training efficiency for diverse models (which may have a non-accuracy performance criterion like hit-rate or loss), we devise a metric called *Area-Under-Performance-Curve* (AUPC). AUPC shows the average normalized performance[2] attained during the model training process. A higher AUPC value suggests that the model can attain a better performance with less computing cost, thus preferred by general users.

For each model, we conduct a series of measurements under different (learning rate, batch size) setups, and Fig. 1 depicts the relationship between the (GR_T, GR_S) status (at iteration 50, an early training stage) and the ultimate AUPC value. In Fig. 1, for each model there exists a *healthy zone* for (GR_T, GR_S) which can yield a good training performance—this aligns with our previous claim that both an over-large and over-small gradient redundancy would be inefficient.

Moreover, we can observe that that healthy zones across different models are similar: *the ideal values of GR_T are mostly all close to 20, and meanwhile the ideal values of GR_S are mostly within the range of [0.5, 0.65] (we use the average, 0.58, as the default target)*. In fact, such a common healthiness range is not a coincidence but endorsed by our derivative analysis and the industrial practices of OpenAI [41], which will be detailed in Appendix E.

This generality is further confirmed in the temporal dimension by Fig. 2. During the critical learning period, a well-configured run continuously stabilizes around the target GR_T value, whereas misconfigured runs persistently deviate from it and exhibit degraded validation performance. This suggests that the healthiness target remains informative throughout the critical period, rather than being valid only at a single early-stage snapshot. In addition, the GR metrics are computed from the vanilla gradient, prior to any optimizer-specific transformations like momentum and smoothing are adopted in advanced optimizers [27, 39, 53]—making the healthiness criterion a shared objective. Different optimizers essentially express different strategies for reaching that healthy state, rather than altering what the state is.

Based on the above analysis, we learn that the gradient redundancy metrics can play as a generic gauge on the healthiness (efficiency) of a given hyper-parameter configuration. Next we show how to leverage such a gradient-redundancy gauge (GR-Gauge) to attain efficient HPO.

4. Solution: HPO with GR-Gauge

In this section, we propose GR-Gauge, a general method that helps to efficiently set up the learning rate and batch size in training by gauging the gradient redundancy.

4.1. Correlation Analysis

While GR_T and GR_S can help judge whether a training state is healthy or not, it is still unclear how to approach the healthy state, because GR_T and GR_S themselves cannot be directly adjusted. Fortunately, we find that there exists a monotonic correlation between (GR_T, GR_S) and (learning rate, batch size), expressed by the following theorems:

[2]Some models may have non-accuracy performance metrics like loss or F1 score [48]. In each case, we map the initial metric value to 0 and map the best metric value (best one ever appeared under any setup) to 100%; the instantaneous training performance is normalized by such a range.

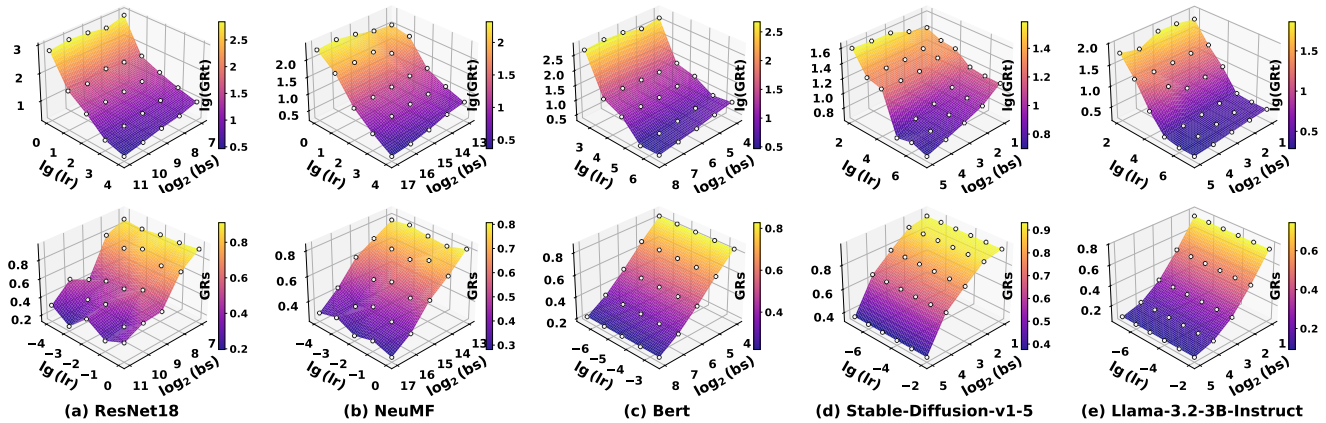


Figure 3. Explore the expected GR_T and GR_S values (at iteration-50 which represents the early training stage) with different (learning rate, batch size) settings, for all models described in Table 1. **Row 1**: GR_T metric value. **Row 2**: GR_S metric value.

Theorem 1. *There exists a positive partial derivative relationship between the learning rate η and the temporal gradient redundancy GR_T:*

$$\frac{\partial}{\partial \eta}\mathbb{E}[GR_T] > 0. \tag{3}$$

Theorem 2. *There exists a negative partial derivative relationship between the batch size B and the spatial gradient redundancy GR_S:*

$$\frac{\partial}{\partial B}\mathbb{E}[GR_S] < 0. \tag{4}$$

The proofs of Theorem 1 and Theorem 2 can be found in Appendix B and Appendix C. We further empirically measure their relationship with a series of testbed measurements as shown in Fig. 3, which also confirm the existence of such correlations. Next, we show how to leverage such correlations to optimize HPO efficiency with the guidance of GR_T and GR_S.

4.2. (GR_T, GR_S)-guided HPO

Based on the target healthiness range of (GR_T, GR_S) and the above correlation, we can act more wisely in HPO. Specifically, we note that gradient redundancy can help optimize HPO efficiency in three aspects. First, they can tell when to stop an ongoing trial. Second, they can suggest a more promising candidate configuration to search next. Third, they can dictate whether the aborted training state can be reused by the next trial—an optimization angle rarely explored before.

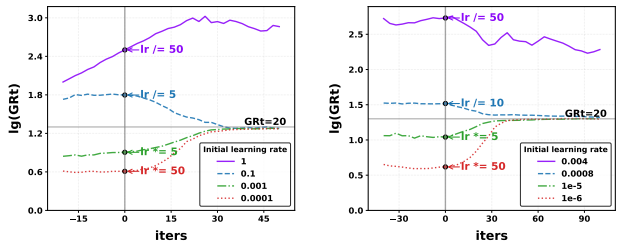
Optimization-1: GR-guided scheduling (early-stopping). We first attain efficient early-stopping by periodically checking GR_T and GR_S. Specifically, we keep[3] checking GR_T and GR_S once after every k (defaulted[4] to 5) iterations; if their values are out of the healthy range, the current trial can be viewed as failed and we need to move on to the next trial. This way, we can timely identify improper configurations. Note that we skip the first 20 iterations in each new trial to confront gradient fluctuations at training commencement.

Optimization-2: GR-guided searching. We then estimate the next configuration to search in HPO based on the gap between the current value of (GR_T, GR_S) and the target healthiness values $(GR_T^*, GR_S^*) = (20, 0.58)$. We adjust the learning rate upward when GR_T is below its target and vice versa; similarly, we adjust the batch size downward when GR_S exceeds its target value and vice versa. As shown in Algorithm 1, in practice we scale up the learning rate by $(GR_T^*/GR_T)^{1/\xi_T}$, where ξ_T controls the aggressiveness of adjustment (defaulted to 3); we scale up the batch size by $(GR_S/GR_S^*)^{1/\xi_S}$, where ξ_S controls the aggressiveness of adjustment (defaulted to 1).

Optimization-3: GR-guided state-reusing. Additionally, we note that in many cases the next trial in HPO can reuse the intermediate training states in previous trials. As shown in Fig. 4 and Fig. 5, if we switch to another configuration without resetting the training states, in many cases (but not always) the (GR_T,GR_S) values can still be remediated to the healthy state, and we further find the ultimate training accuracy is still similar to that with restarted training (as endorsed by our later evaluation). Therefore, when switch-

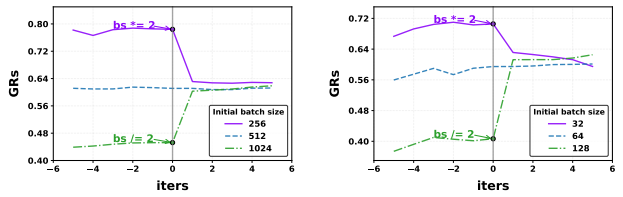
[3]We only check healthiness in the critical period—identified with the same gradient-norm based method as in [58], where the detection threshold is set to 0.01, and the typical critical period takes up around 10% of the total iterations. Critical-period is sufficient for shaping the training process into a good mode, and classical optimizers like AdamW [39] can usually work well in handling the remaining phases.

[4]While several hyper-parameters like k are newly-induced by GR-Gauge, it is much less model-sensitive. As shown later in our evaluations (Fig. 7), the default hyper-parameters of GR-Gauge we identified can generally work well across diverse models, which are different with the learning rate and batch size hyper-parameters that should be made model-specific.



(a) ResNet (b) Bert

Figure 4. Trajectory of GR_T if reusing the aborted state when switching to a new learning rate setup.



(a) ResNet (b) Bert

Figure 5. Trajectory of GR_S if reusing the aborted state when switching to a new batch size setup.

ing to another configuration, we can try to inherit the training state in the aborted trial (i.e., continue training)—by checking whether the GR_T and GR_S metric value can be restored to the healthy state after an exemption period (defaulted to 20 iterations). If not—suggesting that the model has been irreversibly damaged by improper setup—we still restart training from scratch. This way, we can avoid wasting the previous training efforts and make HPO more efficient. Furthermore, while state reuse may slightly affect the final model quality, we show in Sec. 5.4 that the resulting gap is minor and acceptable in practice.

Overhead Reduction. While GR-Gauge no longer requires costly validation tests, it still brings overhead in calculating the metric value. We adopt two techniques to reduce such cost. First, we calculate GR_S at *per-device* instead of *per-sample* granularity, which can preserve the calculation validity while substantially reducing the computation cost. The details are elaborated in Appendix D. Second, we calculate GR_T and GR_S with a set of sampled (totally $L = 5 \times 10^7$) parameters instead of the full model. Here the parameters are sampled from each layer proportionally to its size; existing works [29, 40] show that parameters within the same model layer usually share the same gradient statistical patterns.

5. Evaluation

We conduct experiments in a 16-node cluster with totally 64 NVIDIA GeForce RTX 2080 Ti GPUs. With that cluster we train the models from diverse tasks as introduced in Table 1. The hyper-parameter space of learning rate and batch size is

Algorithm 1: (GR_T, GR_S)-guided HPO

Input : $i, lr, bs, GR_T^*, GR_S^*$ ▷ i: current iteration; lr: learning rate; bs: batch size; $GR_T^* = 20$, $GR_S^* = 0.58$: target metric values; default hyperparameters $k = 5, T = 20, \xi_T = 3, \xi_S = 1$

if $i \pmod k = 0$ **and** $i > T$ **then**
  calculate GR_T, GR_S based on Eq. 1 and Eq. 2
  if $|\frac{GR_T - GR_T^*}{GR_T^*}| > 0.1$ **then**
    $lr \leftarrow lr \times (GR_T^*/GR_T)^{1/\xi_T}$ ▷ If GR_T falls around the expected value, do nothing; else perform reconfiguration.
  if $|\frac{GR_S - GR_S^*}{GR_S^*}| > 0.1$ **then**
    $bs \leftarrow bs \times (GR_S/GR_S^*)^{1/\xi_S}$ ▷ If GR_S falls around the expected value, do nothing; else perform reconfiguration.
  if *GR_T, GR_S not return to healthy state* **then**
    restart training with new lr and bs

Table 2. Hyper-parameter Search Space

Model	learning rate	batch size
ResNet18	$\{10^k\}_{k=-6}^{0}$	$\{2^k\}_{k=6}^{12}$
NeuMF	$\{10^k\}_{k=-6}^{0}$	$\{2^k\}_{k=11}^{18}$
Bert	$\{a \times 10^k\}_{k=-7,a\in\{2,5\}}^{-3}$	$\{2^k\}_{k=3}^{6}$
Stable-diffusion	$\{10^k\}_{k=-7}^{-2}$	$\{2^k\}_{k=1}^{5}$
Llama-3.2-3B	$\{10^k\}_{k=-7}^{-1}$	$\{2^k\}_{k=1}^{5}$

listed in Table 2. For all the models, we adopt the AdamW optimizer [39]. Besides, we include five baselines[5]: *Random Search* (RS), *Successive halving* (SH), *Hyperband* (HB), BOHB, and *cost-frugal optimization* (CFO) [56].

5.1. End-to-End Performance

Metrics. To evaluate the cost-efficiency of GR-Gauge, we set multiple validation targets and record the GPU-time (absolute time multiplied by the GPU number) required to reach each target. The targets are set close to the optimal validation metric values ever attained under each model-dataset combination. Besides, we also experiment with fixed GPU-time budget and record the validation performance achieved within each budget level. By setting multiple validation targets (aligned with the same method as in Fig. 1) or budgets, we seek to cover diverse cost-accuracy trade-off levels. To mitigate the randomness, we repeat each experiment for 5

[5]The former four are elaborated in Sec. 2.2; CFO represents a continuous online-tuning method based on hyper-gradients, covering a design point absent in the others. Methods in popular HPO frameworks (e.g., Ray Tune [35]) are incremental variants of these core algorithms—ASHA [31] from SH, PBT [23] from HyperBand, and Optuna [7] from BOHB—and are thus covered by our baseline set.

Table 3. GPU-Time (normalized by GR-Gauge) required to reach different validation targets by different HPO methods. "N/A" indicates that the target is not reached within a reasonable time budget.

Model	Validation Targets	*RS*	*SH*	*HB*	*BOHB*	*CFO*	*GR-Gauge*
ResNet18	75%	6.42×	7.50×	7.28×	6.64×	3.23×	**504s**
	85%	9.13×	8.56×	7.81×	7.53×	3.54×	**977s**
	89%	6.27×	6.02×	5.47×	4.23×	3.11×	**2010s**
NeuMF	55%	2.44×	0.69×	0.65×	**0.61**×	8.47×	275s
	60%	3.81×	1.17×	1.16×	1.03×	10.6×	**325s**
	68%	N/A	N/A	3.24×	2.75×	N/A	**426s**
Bert	70%	17.8×	2.08×	2.16×	0.82×	4.44×	**3650s**
	80%	24.8×	7.47×	6.96×	6.52×	6.37×	**5090s**
	85%	18.5×	9.27×	7.86×	7.95×	3.32×	**11000s**
Diffusion	85%	1.60×	1.61×	1.53×	1.51×	1.96×	**829s**
	90%	N/A	3.33×	3.44×	3.40×	2.24×	**1640s**
	92%	N/A	3.55×	3.61×	2.50×	2.08×	**2350s**
Llama3	50%	6.26×	2.88×	2.81×	**0.12**×	4.33×	21500s
	55%	3.98×	2.22×	1.83×	1.79×	2.68×	**53400s**
	59%	N/A	2.62×	1.83×	1.30×	2.56×	**75300s**

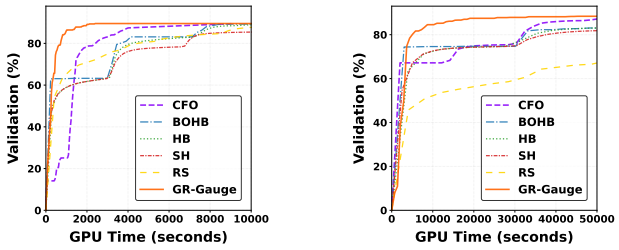
Table 4. Validation performance (in percentage) attained under fixed budgets (GPU-Time) by different HPO methods.

Model	Time Budgets	*RS*	*SH*	*HB*	*BOHB*	*CFO*	*GR-Gauge*
ResNet18	1000s	63.50	58.93	58.08	63.09	25.09	**85.64**
	2000s	69.71	61.74	61.72	63.25	78.58	**88.48**
	4000s	76.95	76.04	79.37	81.03	87.36	**89.51**
NeuMF	200s	50.20	55.76	55.54	**56.94**	41.14	36.18
	500s	54.05	61.15	61.27	61.51	45.22	**68.77**
	1000s	57.80	62.95	66.12	67.53	46.93	**69.11**
Bert	10000s	50.93	71.13	71.34	74.60	67.17	**84.56**
	20000s	55.14	74.15	74.27	74.72	74.45	**87.47**
	40000s	64.33	80.16	82.02	82.28	85.76	**88.19**
Diffusion	1000s	84.41	84.22	82.02	81.62	84.88	**88.60**
	5000s	89.21	89.14	88.98	89.39	92.03	**92.79**
	10000s	89.36	93.76	92.24	93.32	92.19	**94.17**
Llama3	20000s	35.31	41.34	40.04	**52.95**	37.60	42.45
	50000s	39.49	48.97	48.67	53.11	37.60	**54.33**
	100000s	44.97	54.14	55.53	60.54	50.01	**60.81**

times and present the average performance value.

Results. Table 3 shows the results under multiple validation targets. In reaching each validation target, we can see that GR-Gauge requires the shortest training time in most cases. Taking ResNet18 for example, to achieve an accuracy of 85%, GR-Gauge can save the time by 63.6% when compared to BOHB (the second best). Table 4 further lists the results under multiple time budgets. It shows that GR-Gauge can make the most prompt enhancement on validation performance: after 20,000s, the F1 score of the Bert model is already 87.47% under GR-Gauge, yet under other methods they are all below 75%. The two experiments can well demonstrate the superiority of GR-Gauge in time-to-accuracy performance.

In Fig. 6, we further depict the instantaneous training performance (i.e., the best validation-metric value ever appeared under any trial of an HPO process) under different HPO methods. It can be observed that GR-Gauge can rapidly attain a high validation performance within a short time. It suggests that a highly cost-sensitive user can attain a satisfiable training performance under GR-Gauge.



(a) ResNet18 (b) Bert

Figure 6. Curves of the best-ever validation performance under different HPO methods.

Table 5. Training time (s) under GR-Gauge variants.

Model & Validation Targets	no-*R*	no-*RS*	no-*RT*	*GR-Gauge*
ResNet18 [89%]	1.17×	1.46×	1.96×	2010s
NeuMF [68%]	1.27×	2.16×	2.70×	426s
Bert [85%]	3.39×	4.45×	7.95×	11000s
Diffusion [95%]	1.06×	1.10×	1.31×	6620s
Llama3 [59%]	2.32×	4.71×	2.47×	75300s

5.2. Ablation and Overhead Study

Note that GR-Gauge applies healthiness guidance in three aspects: state-reuse, configuration-search and early-termination. We now check the necessity of healthiness guidance in each aspect. To that end, we introduce three degraded versions of GR-Gauge. In no-*R*, we disable state-reuse; in no-*RS*, we disable both state-reuse and healthiness-guided configuration-searching; in no-*RT*, we disable both state-reuse and healthiness-guided early-termination. Table 5 shows the results of the ablation study. It can be seen that all degraded versions require more time than standard GR-Gauge to reach the designated performance target. Meanwhile, no-*RS* and no-*RT* are both less efficient than no-*R*. Therefore, it is indeed necessary to apply healthiness guidance in each of the three aspects.

5.3. Sensitivity Analysis

As displayed in Alg. 1, GR-Gauge introduces several hyper-parameters: k, T, ξ_T, ξ_S; here we conduct sensitivity analysis on them. As shown in Fig. 7, we respectively set k, T, ξ_T, ξ_S to different values, and record the resultant GPU time to reach the validation target (normalized by that under our default setup). On the one hand, such GR-Gauge hyper-parameters do affect the overall performance. For example, over-frequent metric checking (e.g., $k = 1$) increases the computation overheads, while insufficient checking (e.g., $k = 50$) hurts the search efficiency. On the other hand, those

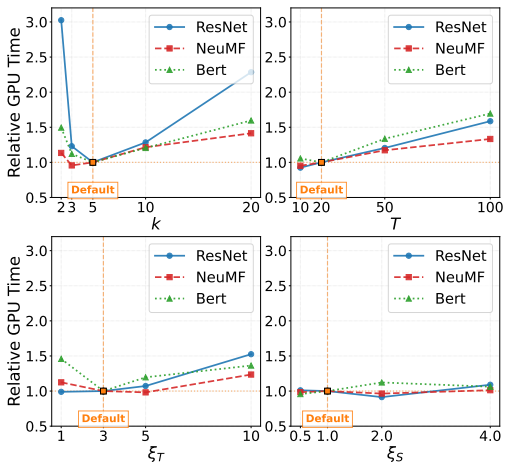


Figure 7. Relative GPU Time required for models to reach target performance (ResNet18: 85% Accuracy; NeuMF: 60% Hit rate; Bert: 80% F1 Score) under different GR-Gauge hyper-parameters.

GR-Gauge hyper-parameters are much less model-sensitive: the default setup we identified works relatively well across different models; GR-Gauge users thus do not need to tune such GR-Gauge hyper-parameters—different from learning rate and batch size which are model-specific.

5.4. Final Accuracy Under State-Reusing

Table 6. Final validation performance of GR-Gauge and from-scratch training under the best-found configuration.

Method	ResNet	NeuMF	Bert	Diffusion	Llama3
From scratch	90.75	69.86	88.40	95.75	60.54
GR-Gauge	89.51	69.11	88.19	94.17	60.81

Table 6 compares the final validation performance of GR-Gauge against training each model from scratch under the best-found hyperparameter configuration. The gaps are minor across all models, confirming that state-reusing does not cause irreversible damage to model quality. The small accuracy trade-off is acceptable given the substantial reduction in training cost.

5.5. Overheads

Table 7 illustrates the breakdown of GR-Gauge running time, showing the GR_T and GR_S overheads. Compared to the total running time, the overheads to maintain the two metrics are indeed not large (commonly less than 4%). Compared to the training speedup with GR-Gauge, such overheads are indeed acceptable.

6. Additional Related Works and Discussions

In recent years, more advanced HPO algorithms have emerged beyond those discussed in Sec. 2.2. These methods focus on diverse aspects like ultra-large scale training configuration [34, 59], federated HPO [60], human-centered interactive HPO [18], and trial dataset retrenchment [26]. These methods did not consider exploiting the internal gradient signal to guide efficient HPO, being orthogonal to us. Note that GR-Gauge is also different from the hyper-gradient based methods [43, 45], because the latter are essentially conducting online-tuning, not suitable for prompt hyper-parameter initialization; meanwhile, they incur large (usually doubled) computation overhead to calculate the hyper-gradients.

Table 7. Time breakdown of the GR-Gauge running time.

Model	**Validation Targets**	**GR_T Overhead(s)**	**GR_S Overhead(s)**	**Total GPU-Time (s)**
ResNet18	89%	56 (2.78%)	28.1 (1.40%)	2013
NeuMF	68%	2.72 (0.63%)	1.11 (0.26%)	426
Bert	85%	216 (1.96%)	346 (3.15%)	11230
Diffusion	95%	159 (2.41%)	220 (3.32%)	6622
Llama3	59%	463 (0.61%)	510 (0.62%)	75320

Meanwhile, our paper does not involve AutoML aspects like model architecture search [36], for which existing HPO methods can work well (note that our viewpoint is that, for learning rate and batch size, those HPO methods are however over-general). That said, our method can be extended to other system hyper-parameters like momentum [53] and synchronization interval [55]. Since these hyper-parameters also influence the gradient redundancy, we can similarly optimize their configurations with a healthiness gauge. In addition, as elaborated in Sec. 3.2, our GR-Gauge method, which is based on vanilla gradients, is potentially applicable to other optimizers besides SGD and AdamW. Moreover, since GR-Gauge is confined to the critical learning period, it can naturally coexist with learning rate schedulers (e.g., cosine decay [38]) and other online-tuning methods, which typically take effect in the subsequent training phases.

7. Conclusion

In this paper, we propose GR-Gauge, a cost-efficient method to set learning rate and batch size for general models. We view model training as a gradient-voting process, and find that for efficiency the temporal and spatial gradient redundancy level should be maintained within a healthy range. By comparing the runtime gradient redundancy with the healthy range, GR-Gauge can guide HPO decisions in searching, scheduling and state-reusing. Extensive experiments demonstrate that GR-Gauge can help achieve cost-effective model training across a diverse range of models.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (NSFC), under grants 62202300, 62572304 and 62232011. Chen Chen is the corresponding author.

References

[1] Ibm watson machine learning. https://www.ibm.com/products/watsonx-ai, 2025. 1, 2

[2] Automatic model tuning with sagemaker ai. https://docs.aws.amazon.com/sagemaker/latest/dg/automatic-model-tuning.html, 2025. 1, 2

[3] Azure machine learning. https://azure.microsoft.com/en-us/products/machine-learning, 2025. 1, 2

[4] Small but mighty: The rise of personal ai model training. https://www.linkedin.com/pulse/beginners-guide-building-ai-model-diana-wolf-torres-wxejc/, 2025. 1

[5] Model training with aws sagemaker. https://docs.aws.amazon.com/sagemaker/latest/dg/train-model.html, 2025. 1, 2

[6] Democratising ai: How startups can train their own large language models. https://medium.com/@PropagenAI/democratising-ai-how-startups-can-train-their-own-large-language-models-ac930b473f28, 2025. 1

[7] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, page 2623–2631, New York, NY, USA, 2019. Association for Computing Machinery. 6

[8] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.*, 13(null): 281–305, 2012. 1, 2

[9] Bernd Bischl, Martin Binder, Michel Lang, Tobias Pielok, Jakob Richter, Stefan Coors, Janek Thomas, Theresa Ullmann, Marc Becker, Anne-Laure Boulesteix, et al. Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 13(2):e1484, 2023. 1, 2

[10] Eole Cervenka. Naruto blip captions. https://huggingface.co/datasets/lambdalabs/naruto-blip-captions/, 2022. 3

[11] Hyunghun Cho, Yongjin Kim, Eunjung Lee, Daeyoung Choi, Yongjae Lee, and Wonjong Rhee. Basic enhancement strategies when using bayesian optimization for hyperparameter tuning of deep neural networks. *IEEE access*, 8:52588–52608, 2020. 1, 2

[12] Aditya Devarakonda, Maxim Naumov, and Michael Garland. Adabatch: Adaptive batch sizes for training deep neural networks. *arXiv preprint arXiv:1712.02029*, 2017. 2

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. 3

[14] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011. 3

[15] Stefan Falkner, Aaron Klein, and Frank Hutter. Bohb: Robust and efficient hyperparameter optimization at scale. In *International conference on machine learning*, pages 1437–1446. PMLR, 2018. 2

[16] Oleg Filatov, Jan Ebert, Jiangtao Wang, and Stefan Kesselheim. Time transfer: On optimal learning rate and batch size in the infinite data limit. *arXiv preprint arXiv:2410.05838*, 2024. 2

[17] Eduardo C Garrido-Merchán and Daniel Hernández-Lobato. Dealing with categorical and integer-valued variables in bayesian optimization with gaussian processes. *Neurocomputing*, 380:20–35, 2020. 2

[18] Joseph Giovanelli, Alexander Tornede, Tanja Tornede, and Marius Lindauer. Interactive hyperparameter optimization in multi-objective problems via preference learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 12172–12180, 2024. 8

[19] F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015. 3

[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE CVPR*, 2016. 3

[21] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *ACM WWW*, 2017. 3

[22] Touseef Iqbal and Shaima Qureshi. The survey: Text generation models in deep learning. *Journal of King Saud University-Computer and Information Sciences*, 34(6):2515–2528, 2022. 1

[23] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. Population based training of neural networks, 2017. 6

[24] Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Artificial intelligence and statistics*, pages 240–248. PMLR, 2016. 1, 2

[25] Tyler Johnson, Pulkit Agrawal, Haijie Gu, and Carlos Guestrin. Adascale sgd: A user-friendly algorithm for distributed training. In *International conference on machine learning*, pages 4911–4920. PMLR, 2020. 2

[26] Arlind Kadra, Maciej Janowski, Martin Wistuba, and Josif Grabocka. Scaling laws for hyperparameter optimization. *Advances in Neural Information Processing Systems*, 36:47527–47553, 2023. 8

[27] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 3, 4

[28] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 3

[29] Sunwoo Lee, Tuo Zhang, and A Salman Avestimehr. Layer-wise adaptive model aggregation for scalable federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8491–8499, 2023. 6

[30] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018. 1, 2

[31] Liam Li, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzur, Moritz Hardt, Benjamin Recht, and Ameet Talwalkar. A system for massively parallel hyperparameter tuning. In *Proceedings of Machine Learning and Systems*, pages 230–246, 2020. 6

[32] Mu Li, Tong Zhang, Yuqiang Chen, and Alexander J Smola. Efficient mini-batch training for stochastic optimization. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 661–670, 2014. 2

[33] Yinglong Li. Research and application of deep learning in image recognition. In *2022 IEEE 2nd international conference on power, electronics and computer applications (ICPECA)*, pages 994–999. IEEE, 2022. 1

[34] Yang Li, Yu Shen, Huaijun Jiang, Wentao Zhang, Jixiang Li, Ji Liu, Ce Zhang, and Bin Cui. Hyper-tune: Towards efficient hyper-parameter tuning at scale. *arXiv preprint arXiv:2201.06834*, 2022. 8

[35] Richard Liaw, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E. Gonzalez, and Ion Stoica. Tune: A research platform for distributed model selection and training, 2018. 6

[36] Ming Lin, Pichao Wang, Zhenhong Sun, Hesen Chen, Xiuyu Sun, Qi Qian, Hao Li, and Rong Jin. Zen-nas: A zero-shot nas for high-performance image recognition. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 347–356, 2021. 8

[37] Jiesong Liu, Feng Zhang, Jiawei Guan, and Xipeng Shen. Uq-guided hyperparameter optimization for iterative learners. *Advances in Neural Information Processing Systems*, 37:386–415, 2024. 2

[38] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with restarts. *CoRR*, abs/1608.03983, 2016. 8

[39] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2017. 2, 4, 5, 6

[40] Na Lv, Zhi Shen, Chen Chen, Zhifeng Jiang, Jiayi Zhang, Quan Chen, and Minyi Guo. Fedca: Efficient federated learning with client autonomy. In *Proceedings of the 53rd International Conference on Parallel Processing*, pages 494–503, 2024. 6

[41] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018. 4

[42] Meta. meta-llama/llama-3.2-3b-instruct, 2024. Model License: Meta Llama 3 Community License. 3

[43] Paul Micaelli and Amos Storkey. Gradient-based Hyperparameter Optimization Over Long Horizons. In *NeurIPS*, 2021. 8

[44] Mlabonne. mlabonne/finetome-100k, 2024. Dataset License: Apache-2.0. 3

[45] Fabian Pedregosa. Hyperparameter optimization with approximate gradient. In *International conference on machine learning*, pages 737–746. PMLR, 2016. 8

[46] Zhongyi Pei, Zhiyao Cen, Yipeng Huang, Chen Wang, Lin Liu, Philip Yu, Mingsheng Long, and Jianmin Wang. Bttackler: A diagnosis-based framework for efficient deep learning hyperparameter optimization. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2340–2351, 2024. 2

[47] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021. 2

[48] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016. 3, 4

[49] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022. 3

[50] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2015. 2

[51] SL Smith. Don't decay the learning rate, increase the batch size. *arXiv preprint arXiv:1711.00489*, 2017. 2

[52] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012. 2

[53] Ilya Sutskever, James Martens, and George Dahl. On the importance of initialization and momentum in deep learning. In *ICML*, 2013. 4, 8

[54] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017. 1

[55] Jianyu Wang and Gauri Joshi. Adaptive Communication Strategies to Achieve the Best Error-Runtime Trade-off in Local-Update SGD. In *SysML*, 2019. 8

[56] Qingyun Wu, Chi Wang, and Silu Huang. Frugal optimization for cost-related hyperparameters. In *Proceedings of the AAAI conference on artificial intelligence*, pages 10347–10354, 2021. 6

[57] Gang Yan, Hao Wang, and Jian Li. Seizing critical learning periods in federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8788–8796, 2022. 2

[58] Gang Yan, Hao Wang, Xu Yuan, and Jian Li. Criticalfl: A critical learning periods augmented client selection framework for efficient federated learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2898–2907, 2023. 2, 5

[59] Peifeng Yu, Jiachen Liu, and Mosharaf Chowdhury. Fluid: Resource-aware hyperparameter tuning engine. *Proceedings of Machine Learning and Systems*, 3:502–516, 2021. 8

[60] Yi Zhou, Parikshit Ram, Theodoros Salonidis, Nathalie Baracaldo, Horst Samulowitz, and Heiko Ludwig. Flora: Single-shot hyper-parameter optimization for federated learning. *arXiv preprint arXiv:2112.08524*, 2021. 8